

PIZZA PING

SQL Data Analysis Project

By
ARPAN KHARE



MySQL

JOINS

Subqueries

GROUP BY

Window Functions



PROJECT OVERVIEW



pizza_ping

Database



4

Tables



12

Queries



3 Levels

Basic→Advanced



Project Goal

Analyze pizza sales data to uncover:

- Revenue & profitability patterns
- Best-selling pizza types & sizes
- Peak ordering hours & category trends
- Rankings with Window Functions





DATABASE SCHEMA — pizza_ping

orders

 `order_id`

`order_date`

`order_time`

order_detail

 `order_details_id`

`order_id` (FK)

`pizza_id` (FK)

`quantity`

pizzas

 `pizza_id`

`pizza_type_id` (FK)

`size`

`price`

pizza_types

 `pizza_type_id`

`name`

`category`

`ingredients`



 FK Links: `order_detail` → `orders` (`order_id`) | `order_detail` → `pizzas` (`pizza_id`) | `pizzas` → `pizza_types` (`pizza_type_id`)



Q1 — Total Orders Placed

Q1

Retrieve the total number of orders placed



```
SELECT
    COUNT(order_details_id)
    AS Total_order
FROM
    order_detail;
```

Insight

Counts every row in order_detail to get the total number of individual pizza orders placed.

Query Result

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
Total_order				
	48620			



Q2 — Total Revenue from Sales

Q2

Calculate the total revenue generated from pizza sales.

```
SELECT
    SUM(order_detail.quantity
        * pizzas.price)
    AS Total_Sales
FROM order_detail
JOIN pizzas
    ON order_detail.pizza_id
    = pizzas.pizza_id;
```

Insight

Multiplies quantity × price for each row, then sums across all orders to get total revenue.

Query Result

Result Grid		Filter Rows:	Export:	Wrap Cell Con
	Total_Sales			
▶	817860.0499999993			



Q3 — Highest-Priced Pizza

Q3

Identify the highest-priced pizza.

```
SELECT
    pizza_types.name, pizzas.size, pizzas.price
FROM
    pizza_types
    JOIN
    pizzas
    ON pizza_types.pizza_type_id = pizzas.pizza_type_id
ORDER BY pizzas.price DESC
LIMIT 1;
```

💡 Insight

Sorts all pizzas by price descending and returns only the top row — the most expensive pizza.

Query Result

Result Grid				Filter Rows:	Export:	Wrap C
	name	size	price			
▶	The Greek Pizza	XXL	35.95			



Q4 — Most Common Pizza Size

Q4

Identify the most common pizza size ordered.



```
SELECT pizzas.size,  
       COUNT(*) AS Max_demand  
FROM order_detail  
JOIN pizzas  
  ON order_detail.pizza_id  
     = pizzas.pizza_id  
GROUP BY pizzas.size  
ORDER BY Max_demand DESC  
LIMIT 1;
```



Insight

Groups all orders by pizza size, counts occurrences, and picks the size with the highest demand.

Query Result

Result Grid			Filter Rows:	Export:	W
	size	Max_demand			
▶	L	18526			



Q5 — Top 5 Most Ordered Pizzas

Q5

List the top 5 most ordered pizza types along with their quantities.

```
SELECT pizza_types.Name,  
       COUNT(*) AS Famous_pt  
FROM order_detail  
JOIN pizzas  
    ON order_detail.pizza_id= pizzas.pizza_id  
JOIN pizza_types  
    ON pizzas.pizza_type_id=pizza_types.pizza_type_id  
GROUP BY pizza_types.name  
ORDER BY Famous_pt DESC  
LIMIT 5;
```

💡 Insight

Three-table JOIN to map orders → pizzas → pizza_types, then ranks by order count to get top 5.

Query Result

	Name	Famous_pt
▶	The Barbecue Chicken Pizza	48620
	The California Chicken Pizza	48620
	The Chicken Alfredo Pizza	48620
	The Chicken Pesto Pizza	48620
	The Southwest Chicken Pizza	48620



Q6 — Quantity per Pizza Category

Q6

Join the necessary tables to find the total quantity of each pizza category ordered.



```
SELECT pizza_types.category,  
       SUM(order_detail.quantity) As Total_Quantity  
FROM pizza_types  
JOIN pizzas  
  ON pizzas.pizza_type_id = pizza_types.pizza_type_id  
  
JOIN order_detail  
  ON pizzas.pizza_id = order_detail.pizza_id  
GROUP BY pizza_types.category;
```

💡 Insight

Shows total units sold per category (Classic, Supreme, Chicken, Veggie) using a 3-table JOIN chain.

Query Result

Result Grid			Filter Rows:	Exp
	category	Total_Quantity		
▶	Classic	14888		
	Veggie	11649		
	Supreme	11987		
	Chicken	11050		



Q7 — Orders by Hour of Day

Q7

Determine the distribution of orders by hour of the day



```
SELECT HOUR(order_time) AS Time,  
       COUNT(HOUR(order_time))  
       AS Total_orders  
FROM orders  
GROUP BY HOUR(order_time)  
ORDER BY Time ASC;
```

 **Insight**

Uses MySQL's HOUR() function to extract the hour from order_time and groups by it to find peak hours.

 Query Result

Result Grid			Filter Rows:	Export
	Time	Total_orders		
▶	9	1		
	10	8		
	11	1231		
	12	2520		
	13	2455		
	14	1472		
	15	1468		
	16	1920		
	17	2336		
	18	2399		
	19	2009		
	20	1642		
	21	1198		
	22	663		
	23	28		



Q8 — Category × Size Distribution

Q8

Join relevant tables to find the category-wise distribution of pizzas.

```
SELECT pizza_types.category,  
       pizzas.size,  
       SUM(order_detail.quantity)  
FROM pizza_types  
JOIN pizzas  
  ON pizza_types.pizza_type_id = pizzas.pizza_type_id  
JOIN order_detail  
  ON pizzas.pizza_id = order_detail.pizza_id  
GROUP BY pizza_types.category,  
         pizzas.size  
ORDER BY category, size ASC;
```

💡 Insight

Cross-analysis: shows how many pizzas of each size were sold in each category — finding the winning combo.

Query Result

	category	size	sum(order_detail.quantity)
▶	Chicken	L	4932
	Chicken	M	3894
	Chicken	S	2224
	Classic	L	4057
	Classic	M	4112
	Classic	S	6139
	Classic	XL	552
	Classic	XXL	28
	Supreme	L	4564
	Supreme	M	4046
	Supreme	S	3377
	Veggie	L	5403
	Veggie	M	3583
	Veggie	S	2663



Q9 — Average Pizzas Ordered per Day

Q9

Group the orders by date and calculate the average number of pizzas ordered per day.

```
SELECT AVG(daily_pizza) AS avg_pizza
FROM (
    SELECT Orders.order_date,
           SUM(quantity)
           AS Daily_pizza
    FROM order_detail
    JOIN orders
        ON orders.order_id= order_detail.order_id
    GROUP BY orders.order_date
) AS avg_daily;
```

💡 Insight

Subquery (alias required!) computes daily pizza total, outer query averages it. Classic subquery pattern.

Query Result

Result Grid		Filter Rows:
	avg_pizza	
▶	138.4749	



Q10 — Top 3 Pizza Types by Revenue

Q10

Determine the top 3 most ordered pizza types based on revenue.

```
SELECT pt.name,  
       SUM(od.quantity * p.price)  
       AS Revenue  
FROM Pizza_types pt  
JOIN pizzas p  
  ON pt.pizza_type_id  
     = p.pizza_type_id  
JOIN order_detail od  
  ON p.pizza_id = od.pizza_id  
GROUP BY pt.name  
ORDER BY Revenue DESC  
LIMIT 3;
```

💡 Insight

Full 3-table JOIN computes revenue per pizza type. LIMIT 3 after ORDER BY returns top revenue earners.

Query Result

Result Grid



Filter Rows:

Export:



	name	Revenue
▶	The Thai Chicken Pizza	43434.25
	The Barbecue Chicken Pizza	42768
	The California Chicken Pizza	41409.5

Result 22 x



Q11 — Revenue % Share per Pizza

Q11

Calculate the percentage contribution of each pizza type to total revenue.

```
SELECT Pizza_types.name,  
       ROUND(  
         (SUM(od.quantity * p.price)*100)  
         /  
         (SELECT SUM(od2.quantity*p2.price)  
          FROM order_detail od2  
          JOIN Pizzas p2  
            ON p2.pizza_id = od2.pizza_id  
         ), 2) AS "Percentage_share(%)"  
FROM Pizzas  
JOIN order_detail  
  ON order_detail.pizza_id = Pizzas.pizza_id  
JOIN Pizza_types  
  ON Pizzas.pizza_type_id = Pizza_types.pizza_type_id  
GROUP BY Pizza_types.name;
```

💡 Insight

Correlated subquery in denominator calculates the grand total revenue; outer query computes each pizza's % share.

Query Result

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	name	Percentage_share(%)			
	The Greek Pizza	3.48			
	The Spinach Supreme Pizza	1.87			
	The Green Garden Pizza	1.71			
	The Italian Capocollo Pizza	3.07			
	The Spicy Italian Pizza	4.26			
	The Spinach Pesto Pizza	1.91			
	The Vegetables + Vegetables Pi...	2.98			
	The Southwest Chicken Pizza	4.24			
	The California Chicken Pizza	5.06			
	The Pepperoni Pizza	3.69			
	The Chicken Pesto Pizza	2.04			
	The Big Meat Pizza	2.81			
	The Soppressata Pizza	2.01			
	The Four Cheese Pizza	3.95			
	The Napolitana Pizza	2.95			
	The Calabrese Pizza	1.95			
	The Italian Vegetables Pizza	1.96			
	The Mediterranean Pizza	1.88			
	The Pepper Salami Pizza	3.12			
	The Spinach and Feta Pizza	2.85			
	The Sicilian Pizza	3.78			
	The Chicken Alfredo Pizza	2.07			
	The Pepperoni, Mushroom, and ...	2.3			
	The Brie Carre Pizza	1.42			



Q12 — Top 3 Pizzas per Category (Revenue)

Q12

Determine the top 3 most ordered pizza types based on revenue for each pizza category.

```
SELECT * FROM (
  SELECT *,
    ROW_NUMBER() OVER (
      PARTITION BY category
      ORDER BY Revenue DESC
    ) AS rn
  FROM (
    SELECT pt.name, pt.category,
      SUM(od.quantity*p.price)
      AS Revenue
    FROM Pizza_types pt
    JOIN pizzas p
    ON pt.pizza_type_id=p.pizza_type_id
    JOIN order_detail od
    ON p.pizza_id= od.pizza_id
    GROUP BY pt.name, pt.category
  ) t
) x
WHERE rn <= 3;
```

Insight

ROW_NUMBER() OVER(PARTITION BY category) assigns rank within each category — WHERE rn<=3 keeps top 3 per group.

Query Result

Result Grid					Filter Rows:		Export:	Wrap Cell Content:
	name	category	Revenue	rn				
▶	The Thai Chicken Pizza	Chicken	43434.25	1				
	The Barbecue Chicken Pizza	Chicken	42768	2				
	The California Chicken Pizza	Chicken	41409.5	3				
	The Classic Deluxe Pizza	Classic	38180.5	1				
	The Hawaiian Pizza	Classic	32273.25	2				
	The Pepperoni Pizza	Classic	30161.75	3				
	The Spicy Italian Pizza	Supreme	34831.25	1				
	The Italian Supreme Pizza	Supreme	33476.75	2				
	The Sicilian Pizza	Supreme	30940.5	3				
	The Four Cheese Pizza	Veggie	32265.70000000065	1				
	The Mexicana Pizza	Veggie	26780.75	2				
	The Five Cheese Pizza	Veggie	26066.5	3				



KEY BUSINESS INSIGHTS



Revenue

$SUM(qty \times price)$ across all orders



#1 Pizza

Most expensive & most ordered identified



L & M

Dominant pizza sizes by volume



Peak Hour

$HOUR()$ distribution shows lunch rush



4 Categories

Classic · Supreme · Chicken · Veggie

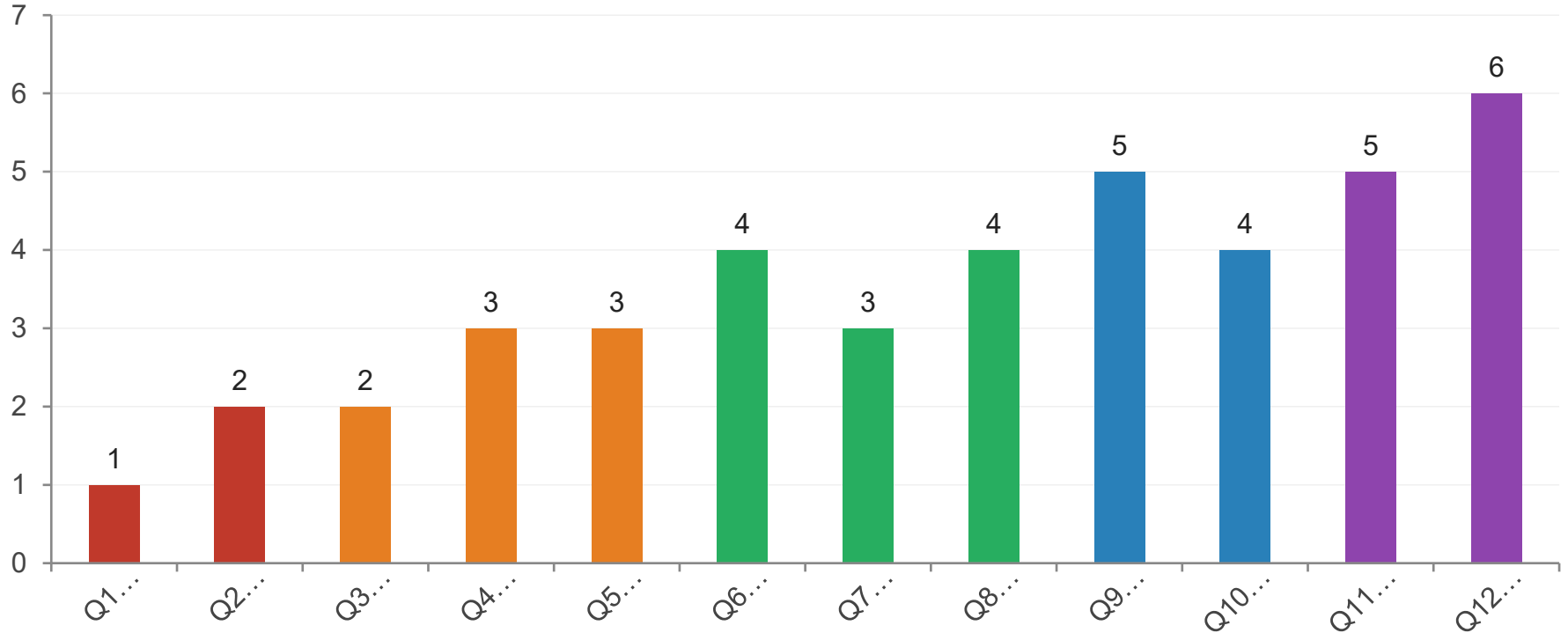


ROW_NUM()

Window fn ranks top 3 per category



QUERY COMPLEXITY MAP



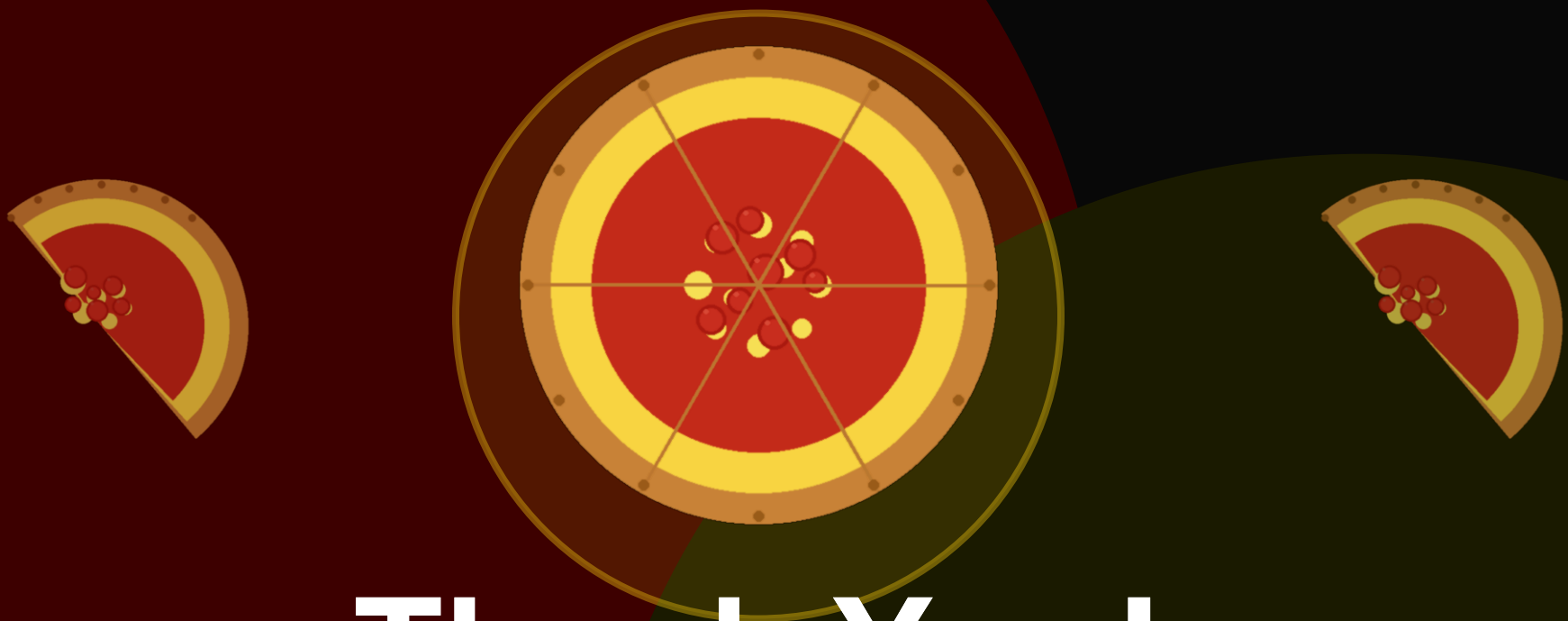
Basic

Intermediate

Joins

Subqueries

Window Functions



Thank You!

Pizza Ping · SQL Data Analysis Project

MySQL · COUNT/SUM · JOINS · GROUP BY · Subqueries · Window Functions · ROW_NUMBER() · PARTITION BY